

UT 01

Valores y variables

Fundamentos de JavaScript: tipos de datos, variables y estructuras básicas

`let` · `const` · `typeof` · `objetos` · `matrices` · `JSON`

¿Qué vamos a ver hoy?

- ▶ 1.1 Ejecución de JavaScript
- ▶ 1.2 Tipos y typeof
- ▶ 1.3 Comentarios
- ▶ 1.4 Declaración de variables
- ▶ 1.5 Identificadores
- ▶ 1.6 Números
- ▶ 1.7 Operadores aritméticos
- ▶ 1.8 Valores booleanos
- ▶ 1.9 null y undefined
- ▶ 1.10 Literales de cadena
- ▶ 1.11 Literales de plantilla
- ▶ 1.12–1.13 Objetos
- ▶ 1.14 Matrices
- ▶ 1.15 JSON
- ▶ 1.16–1.17 Desestructuración

1.1

Ejecución de JavaScript

1

Navegador web

Incrustar `<script>` en HTML y usar `window.alert` para mostrar valores.

2

Consola del navegador

Herramientas de desarrollo (F12) → pestaña Consola: ideal para pruebas rápidas.

3

Node.js (REPL)

Instalar Node.js y ejecutar `node` en la terminal para un bucle lectura-evaluación-impresión.

4

Entorno de desarrollo

VS Code, WebStorm, Eclipse... editar y ejecutar archivos `.js` con `console.log`.

```
// first.js
let a = 6
let b = 7
console.log(a * b)

$ node first.js
42
```

Cuatro formas de ejecutar el mismo código

1.2

Tipos y el operador typeof

number

boolean

null /
undefined

string

symbol

object

Tipos primitivos = todos excepto object

```
typeof 42           // 'number'
typeof true         // 'boolean'
typeof 'hola'       // 'string'
typeof null         // 'object' !
typeof undefined    // 'undefined'
```

⚠ Curiosidad histórica

typeof null es "object", aunque null es un tipo distinto. Es una casualidad histórica del lenguaje.

1.3

Comentarios

- Comentario de una línea: comienza con `//` y llega hasta el final de la línea.
- Comentario de varias líneas: se delimita con `/* ... */`.
- JavaScript no tiene una forma especial de comentarios de documentación (a diferencia de Java).
- Existen herramientas de terceros, como JSDoc, que ofrecen esa funcionalidad.

```
// como este
```

```
/*  
    como  
    así  
*/
```

1.4

Declaración de variables

let

Variable cuyo valor puede cambiar.

```
let counter = 0  
counter = 1
```

const

Variable cuyo valor nunca cambia.

```
const PI = 3.14159  
// PI = 4 -> Error
```

var (evitar)

Forma obsoleta, con comportamiento peligroso.

```
var counter = 0  
coutner = 1 // ¡bug!
```

Regla de oro: declara siempre con let o const, nunca con var.

- Si no inicializas una variable, su valor es undefined.
- Puedes declarar varias variables en una sola instrucción: `const FREEZING = 0, BOILING = 100.`

Identificadores

- Compuestos por letras Unicode, dígitos, `_` y `$`.
- El primer carácter no puede ser un dígito.
- Convención habitual: camelCase (miVariable).
- Evita `$` y `_` al principio o al final del nombre.

- `let counter = 0` ✓ válido
- `let 2fast = true` ✗ no válido
- `const  $\pi$  = 3.14159` ✓ válido (Unicode)

Palabras reservadas

`break, case, class, const, else, false, for, function, if, new, null, return, this, true, typeof, var, while...`

Usables pero no recomendadas

`await, as, async, from, get, of, set, target, yield`

1.6

Números

- No existe un tipo entero explícito: todos son de doble precisión en coma flotante.
- Enteros seguros entre `Number.MIN_SAFE_INTEGER` y `Number.MAX_SAFE_INTEGER`.
- Los valores fraccionarios tienen errores de redondeo inevitables ($0.1 + 0.2$).
- `parseFloat` / `parseInt` convierten cadenas a números.
- `toString()` convierte un número de nuevo a cadena.

```
0.1 + 0.2
// 0.30000000000000004

parseFloat('3.14') // 3.14
parseInt('3')       // 3
(3).toString()      // '3'
5 / 0               // Infinity
0 / 0               // NaN
```

💡 Para euros y centavos: representa las cantidades como múltiplos enteros de un centavo.

1.7

Operadores aritméticos

+ - * /

Suma, resta, multiplicación, división. / siempre da resultado decimal: $1/2 = 0.5$

%

Resto de la división. $3.5 \% 1.2$ es 1.1

**

Elevar a potencia. $2 ** 10$ es 1024

+= , ++ , --

Operaciones combinadas: contador += 10, contador++



```
let counter = 7
let agent = '00' + counter
// agent = '007'

6 * '7' // 42
null + undefined // NaN
```

Consejo

No confíes en las conversiones automáticas de tipos con operadores aritméticos. Prefiere los literales de plantilla a la concatenación de cadenas.

Valores booleanos

El tipo booleano tiene dos valores: false y true. En una condición, cualquier valor se convierte automáticamente a booleano.

false (falsy)

- 0
- NaN
- null
- undefined
- "" (cadena vacía)

true (truthy)

- Cualquier otro número
- Cualquier otra cadena
- Objetos y matrices
- true

1.9

null y undefined

undefined

Valor por defecto de una variable declarada pero no inicializada, o de un parámetro no proporcionado.

```
let x
// x es undefined
```

null

Denota la ausencia intencionada de un valor. Debe asignarse explícitamente.

```
let y = null
```

Consejo

En cualquier proyecto, decide explícitamente un enfoque: usa undefined o null para la ausencia de valor, no ambos indistintamente.

1.10

Literales de cadena

- Se escriben entre comillas simples `' '` o dobles `" "`.
- Para incluir una comilla del mismo tipo, escápala con `\`.
- Secuencias de escape: `\n` (salto de línea), `\t` (tabulación), `\\` (barra invertida)...
- JavaScript usa UTF-16: algunos caracteres (como emojis) ocupan dos unidades de código.

```
'Hola'           // comillas simples
"Hola"           // comillas dobles
'It\\'s ok'       // escapando comilla
'Línea1\\nLínea2'
let g = 'Hello 🌐'
g.length          // 8 (no 7)
```

⚠ El operador `[ ]` no funciona bien con caracteres de dos unidades de código (como emojis).

1.11

Literales de plantilla

- Delimitados por comillas invertidas `` ... ``.
- Pueden contener expresiones dentro de `${ ... }`.
- Pueden abarcar varias líneas de forma natural.
- Se pueden anidar plantillas dentro de `${ ... }`.

```
let destino = 'mundo'
let saludo = `Hola, ${destino}!`

`Hola, ${destino.toUpperCase()}!`
// 'Hola, ¡MUNDO!'

`<div>${destino}</div>`
```

Consejo: prefiere las plantillas literales a la concatenación con +

1.12-1.13

Objetos

- Conjunto de pares nombre/valor ("propiedades").
- No hay encapsulación: todos los datos son públicos.
- Notación de punto para acceder: `harry.age`
- `delete harry.salary` elimina una propiedad.
- Propiedad calculada: `harry[campo.toLowerCase()]`
- Se admite coma final y abreviatura `{ name, age }`.

```
const harry = {  
  name: 'Harry Smith',  
  age: 42,  
}  
  
harry.age = 40  
harry.salary = 90000  
delete harry.salary
```

const impide reasignar la variable, no mutar el objeto.

1.14

Matrices (Arrays)

- Una matriz es un objeto cuyas propiedades son '0', '1', '2'...
- `length` es uno más que el índice más alto.
- Los elementos pueden ser de tipos distintos.
- `typeof` devuelve 'object'; usa `Array.isArray()` para comprobar.
- Se simulan matrices multidimensionales con matrices de matrices.

```
const numbers = [1, 2, 3, 'many']
numbers.length // 4
numbers[1]     // 2

const grid = [
  [16, 3, 2, 13],
  [5, 10, 11, 8]
]
grid[1][2] // 11
```

Formato de texto ligero para intercambiar datos entre aplicaciones.

- Todas las cadenas y nombres de propiedad van entre comillas dobles.
- No se permiten comas finales ni elementos omitidos.
- `JSON.stringify(obj)` → convierte un objeto JS en texto JSON.
- `JSON.parse(texto)` → convierte texto JSON en un objeto JS.

```
{  
  "name": "Harry Smith",  
  "age": 42,  
  "lucky": false  
}
```

```
JSON.stringify(harry)  
JSON.parse(texto)
```

1.16–1.17

Desestructuración

- Extrae valores de matrices u objetos en variables directamente.
- Con matrices: por posición.
- Con objetos: por nombre de propiedad.
- Admite valores por defecto y capturar el resto con ...

Para intercambiar variables:
`[x, y] = [y, x]`

```
// Matrices
let [first, second] = [1, 2]

// Objetos
let { name, age } = harry

// Resto y valor por defecto
let [a, ...rest] = [1, 2, 3]
let { nick = 'None' } = harry
```

Para escribir JavaScript con menos confusión

- 1 Declara las variables con `let` o `const`, nunca con `var`.
- 2 Usa valores booleanos reales para todas las condiciones.
- 3 Decide un único enfoque para `null` vs. `undefined`.
- 4 Prefiere los literales de plantilla a la concatenación con `+`.
- 5 No confíes en las conversiones automáticas de tipos.